

Not So Invisible: Classifying Darknet Traffic from Network Flows



CS4403 Final Report

Group Members

Jaspinder Singh, Nomaan Imran Saiyed, Syed Owais Haider Kazmi

GitHub Repository

<https://github.com/jaspindersingh919/CS4403-Project>

1. Project Overview and Objectives

The goal of this project was to test whether darknet-related traffic can be identified using network flow features. Instead of looking at payload contents, we focused on numerical statistics such as packet counts, byte counts, durations, and rates. This is a realistic setting because privacy-related traffic such as Tor and VPN often hides the packet contents but still leaves measurable behavior in the flow data.

The main question behind the project was straightforward: can network flow features reveal enough information to classify darknet-related traffic accurately? A second question followed from that. If the answer is yes, which models work best, what do they learn, and what limits still remain?

We also wanted the project to be more than one model and one final score. Tor was the smallest class in the dataset, so overall accuracy alone could be misleading. For that reason, we compared a simple baseline model with stronger tree-based models, evaluated Tor separately, used PCA to inspect the feature space, and used clustering to study the data without labels.

The final project is therefore not only about reporting a high classification score. It is also about understanding which methods fit the data best, what the results mean for the minority Tor class, and what additional insight can be gained from visualization and clustering.

2. Dataset and Preparation

The dataset used in this project was CIC-Darknet2020. We chose it because it directly matches the problem we wanted to study: darknet-related traffic classification using flow-level features. It contains both broad traffic labels and more detailed application labels, which allowed us to test more than one prediction task.

After preprocessing, the working dataset contained 141,531 rows and 85 columns. The final version included 83 feature columns and 2 target columns: Label and Label.1.

The Label column was used for the main classification task. It contained four traffic classes:

- Non-Tor
- NonVPN
- VPN
- Tor

The Label.1 column was used for application-level categories such as:

- Browsing
- Chat
- Email
- P2P
- Audio-Streaming
- Video-Streaming
- File-Transfer
- VOIP

One of the biggest challenges in the dataset was class imbalance. Non-Tor was much larger than the other classes, while Tor was the smallest by far. This became important later because a model can still achieve high overall accuracy on imbalanced data while performing worse on the minority class.

Before training, we cleaned the dataset by removing empty unnamed columns, correcting inconsistent labels, handling missing values, converting features to numeric form, and removing identifier-style columns such as Flow ID, IP addresses, ports, timestamp, and protocol. These fields were removed because they could allow the models to memorize identities instead of learning traffic behavior.

We used the CIC-Darknet2020 dataset because it supports two useful views of the problem. The first is broad traffic classification with the labels Non-Tor, NonVPN, VPN, and Tor. The second is application-level classification through Label.1, which includes categories such as Browsing, Chat, Email, P2P, Audio-Streaming, Video-Streaming, File-Transfer, and VOIP.

We then used an 80/20 train-test split with stratified sampling so that class proportions stayed similar in the training and test sets. This was especially important because Tor was underrepresented.

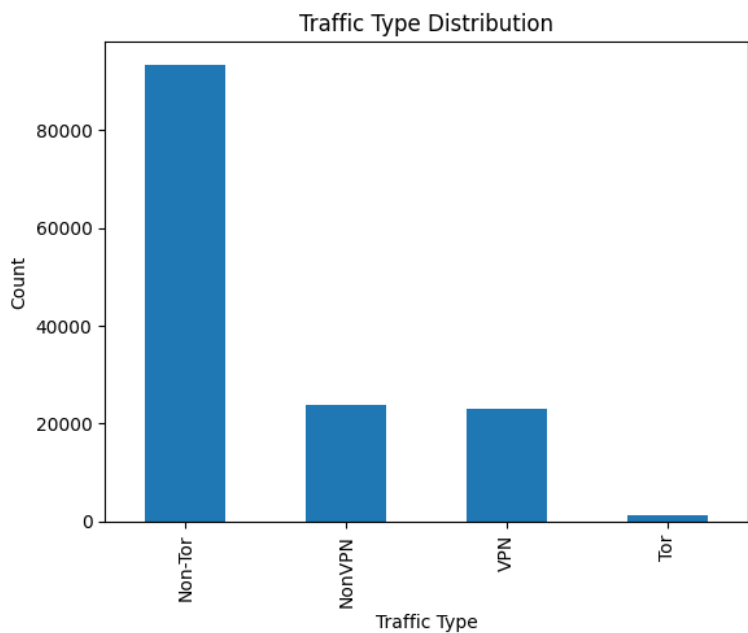


Figure 1. Distribution of the main traffic classes in the cleaned CIC-Darknet2020 dataset. Non-Tor is the dominant class, while Tor is the smallest class.

Table 1 summarizes the main preprocessing steps.

Step	Description	Why it mattered
Remove unnamed columns	Dropped empty columns from the raw CSV	Reduced noise and simplified the dataset
Correct corrupted label	Fixed inconsistent values in Label	Ensured reliable supervision
Standardize Label.1	Cleaned application category names	Made application-level classification valid

Handle missing values	Treated missing Flow Bytes/s values and invalid numbers	Prevented model errors
Remove identifier columns	Excluded Flow ID, IPs, timestamp, ports, and protocol	Reduced memorization and leakage risk
Stratified train-test split	Used an 80/20 split with preserved class proportions	Improved fair evaluation under imbalance

Table 1. Summary of preprocessing steps applied before model training.

3. Methodology

We used several methods so that the project would answer more than one question. The idea was to compare models, test the minority class carefully, and also study the feature space more directly.

4.1 Evaluation Strategy

We evaluated the models using:

- accuracy
- balanced accuracy
- macro F1-score
- class-wise precision and recall
- confusion matrices

We used these measures because accuracy alone is not enough in an imbalanced dataset. Since Tor was the smallest class, we needed metrics that showed how well the minority class was being learned.

4.2 Logistic Regression

Logistic Regression was used as a simple baseline. We included it because it is a standard linear classifier and gave us a useful comparison point. If it performed close to the tree-based models, then the problem might be mostly linear. If it performed much worse, that would suggest more complex structure.

4.3 Random Forest

Random Forest was used as the main baseline classifier. It was a natural choice because it works well on structured tabular data and can capture nonlinear relationships between features. It also provides feature importance values, which made interpretation easier later in the project.

4.4 Class-Weighted Random Forest

We added a class-weighted Random Forest to directly address the Tor imbalance issue. The goal was to see whether giving more weight to the minority class would improve Tor performance. This model was mainly included as an imbalance check rather than as a separate final solution.

4.5 XGBoost

XGBoost was added later as a stronger tree-based ensemble model. Since Random Forest already showed that tree-based methods fit the problem well, XGBoost was used to test whether a boosted model could improve the results even further.

4.6 Label.1 Classification

We also trained a model for the application-level task using Label.1. This extended the project beyond broad traffic-type classification and helped test whether the same flow features could also distinguish application behavior.

4.7 PCA

PCA was used only for visualization. It reduced the high-dimensional feature space to two dimensions so that we could inspect the structure of the data. This helped explain why the linear model performed worse than the stronger tree-based models.

4.8 Clustering

We used K-means and DBSCAN as unsupervised methods. K-means was used to explore whether the data formed natural groups. DBSCAN was used to study density-based structure and outliers. These methods were not meant to replace classification, but to give another view of the data.

4.9 Robustness Checks

We also included robustness checks such as duplicate checks, label shuffling, cross-validation, and a stricter group-based split. These helped us test whether the models were learning real traffic patterns or only benefiting from easier source-specific information.

Method	Type	Purpose	Why it was used
Random Forest	Supervised	Main traffic classification	Strong on tabular nonlinear data
Logistic Regression	Supervised	Baseline comparison	Tests linear separability
Balanced Random Forest	Supervised	Minority-class check	Tests Tor imbalance handling
XGBoost	Supervised	Strong ensemble comparison	Tests boosted tree performance
PCA	Visualization	2D projection of feature space	Helps interpret model behavior
K-means	Unsupervised	Natural grouping	Explores cluster structure
DBSCAN	Unsupervised	Noise and outlier analysis	Detects sparse unusual behavior

Table 2. Summary of methods used in the project.

4. Results

This section summarizes the main results of the project. We first compare the supervised models on the main Label task, then look more closely at Tor, then summarize the Label.1 task, and finally present the PCA, feature-importance, and clustering results.

4.1 Main traffic classification results

For the main Label task, we compared Logistic Regression, Random Forest, class-weighted Random Forest, and XGBoost. The strongest model was XGBoost. It achieved 0.9850 accuracy, 0.9525 balanced accuracy, and 0.9634 macro F1. The baseline Random Forest also performed very strongly with 0.9811 accuracy, 0.9462 balanced accuracy, and 0.9554 macro F1. The class-weighted Random Forest was almost identical to the baseline Random Forest and did not provide a meaningful improvement. Logistic Regression was much weaker overall, which showed that the problem was not well described by a simple linear boundary.

Model	Accuracy	Balanced Accuracy	Macro F1	Tor Precision	Tor Recall	Tor F1
XGBoost	0.9850	0.9525	0.9634	0.9880	0.8921	0.9376
Random Forest baseline	0.9811	0.9462	0.9554	0.9686	0.8885	0.9268
Random Forest balanced	0.9808	0.9458	0.9551	0.9686	0.8885	0.9268
Logistic Regression	0.8393	0.8004	0.7196	0.4211	0.8633	0.5660

Table 3. Final comparison of supervised models on the main Label task.

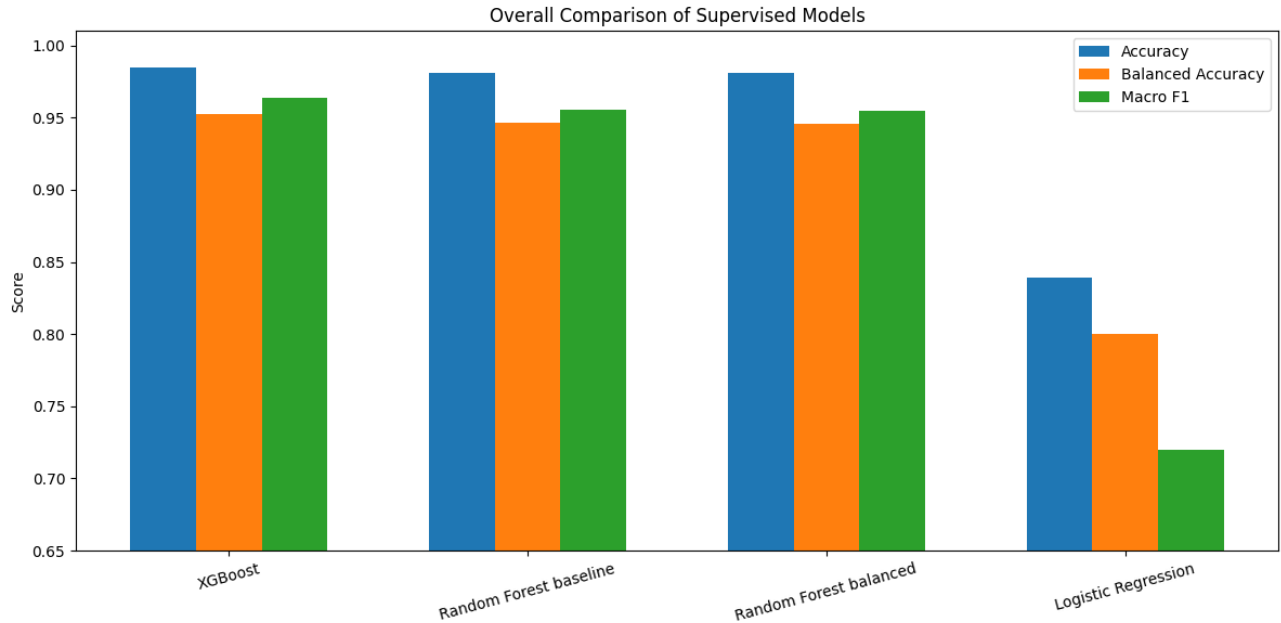


Figure 2. Overall comparison of supervised models using accuracy, balanced accuracy, and macro F1-score. XGBoost gave the strongest overall performance, while Logistic Regression was clearly weaker than the tree-based models.

4.2 Tor-specific results

Tor was the smallest class in the dataset, so we examined it separately. In the XGBoost model, Tor achieved 0.9880 precision, 0.8921 recall, and 0.9376 F1. The baseline and class-weighted Random Forest models produced almost identical Tor results, which showed that simple class weighting did not improve Tor detection in a meaningful way. Logistic Regression had fairly high Tor recall, but very low Tor precision. In practice, that means it labeled many non-Tor flows as Tor and was much less reliable.

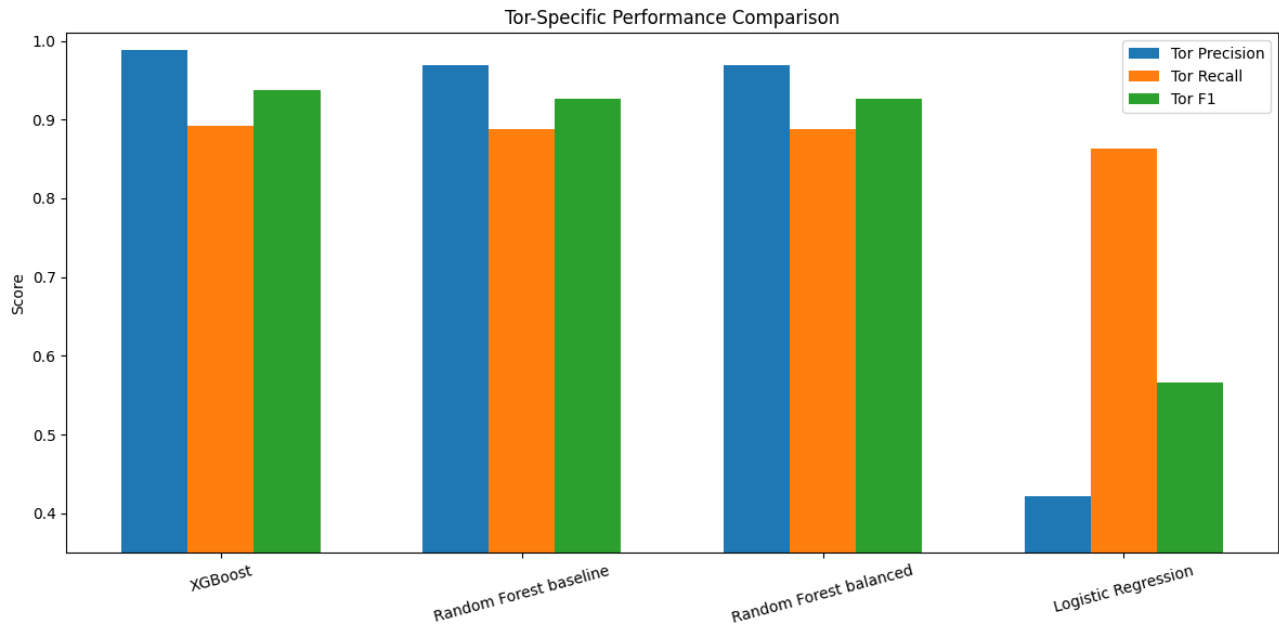


Figure 3. Tor-specific performance comparison across models. The stronger tree-based models maintained strong Tor precision, recall, and F1 despite Tor being the minority class.

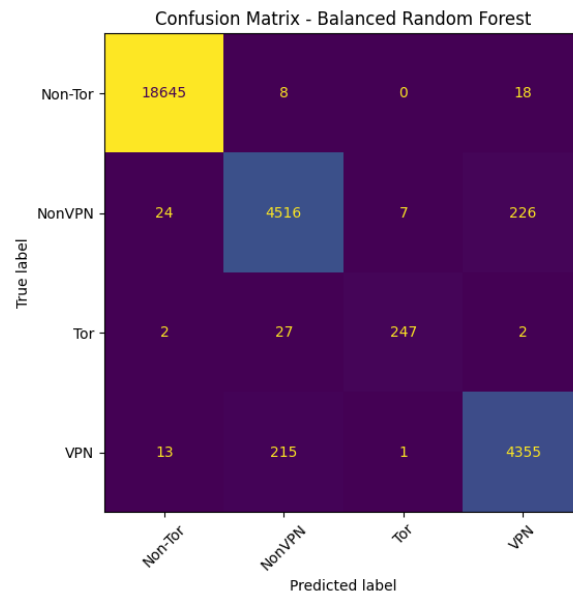


Figure 4. Confusion matrix for the class-weighted Random Forest. Most Tor errors were confused with NonVPN rather than being spread evenly across all classes.

4.3 Application-level classification

We also trained a model to predict Label.1. This task was harder because it involved finer categories such as Browsing, Chat, Email, P2P, and Streaming. The final Label.1 result achieved 87.4% accuracy with an F1-score of 0.780. The confusion matrix showed that P2P and Browsing were predicted well, while Chat, Email, and VOIP were more often confused with each other. There was also overlap between Audio-Streaming and Video-Streaming.

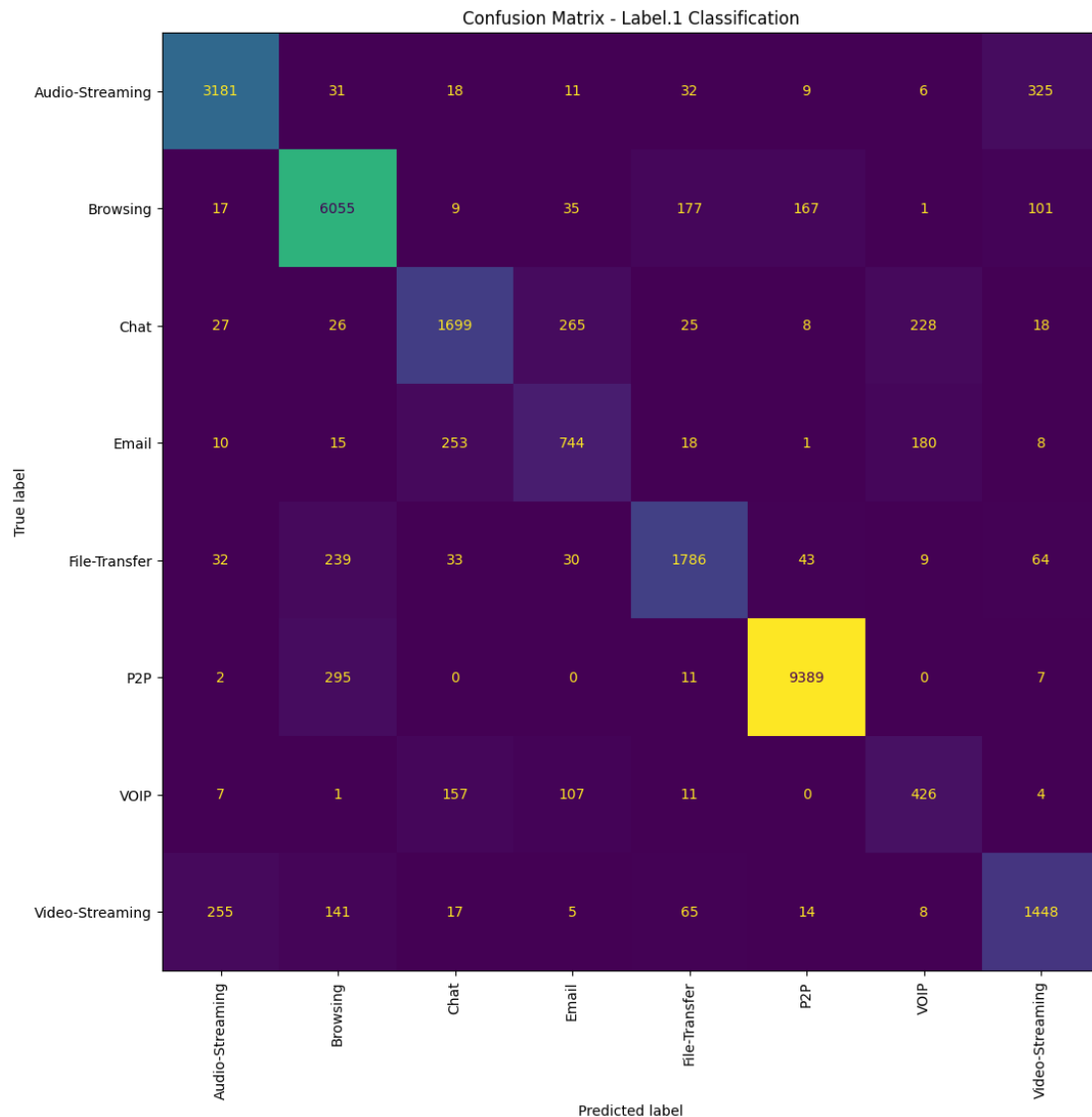


Figure 5. Confusion matrix for the Label.1 classification task. The diagonal cells show correct predictions, while off-diagonal cells show where similar application classes were confused.

4.4 PCA and feature importance

The PCA projection showed that the dataset had visible structure, but the classes were not separated by a clean straight boundary. This helped explain why Logistic Regression performed worse than Random Forest and XGBoost. Feature-importance analysis also showed that the model relied on meaningful flow-level measurements rather than on identifier fields or random variation.

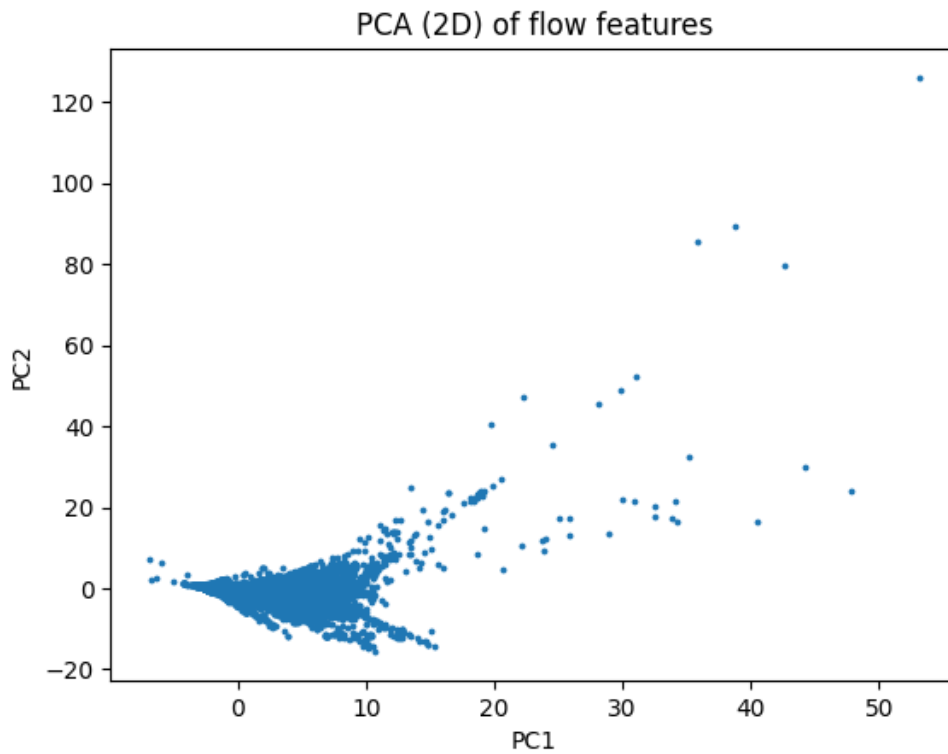


Figure 6. PCA projection of the flow features into two dimensions. The classes show structure, but not a simple linear separation.

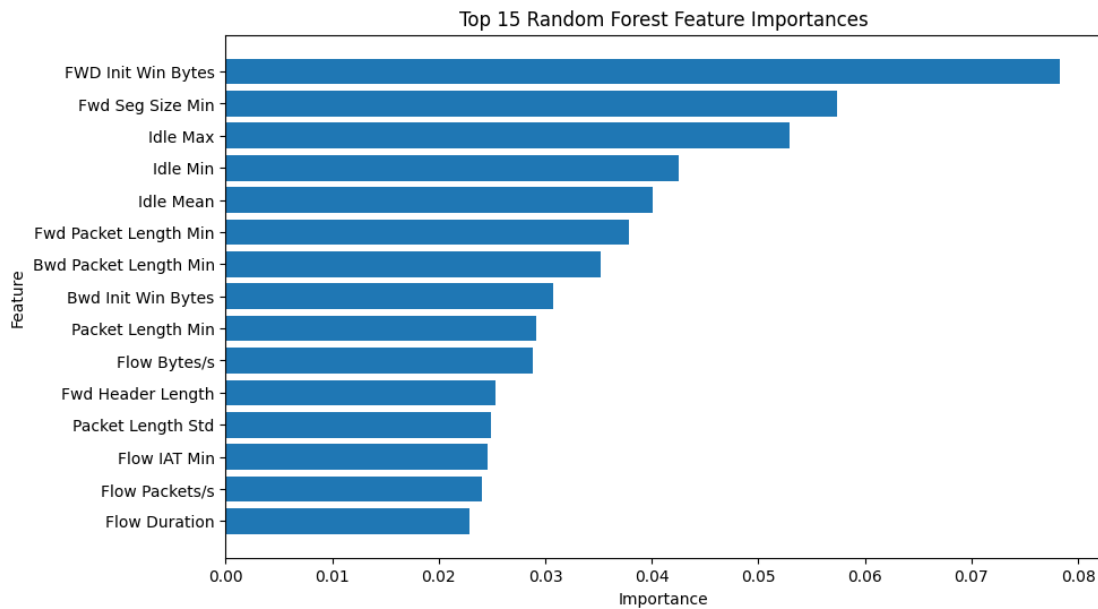


Figure 7. Top 15 Random Forest feature importances. The model relied on flow-level measurements rather than identifier-style columns.

4.5 Clustering results

K-means suggested that the dataset had internal grouping, with the best silhouette result appearing at four clusters. However, those clusters did not perfectly match the true labels. DBSCAN gave a more interesting

result. It identified dense regions along with many noise points, and Tor appeared disproportionately often in those noise points. This suggests that Tor behaves more like an outlier than the other classes.

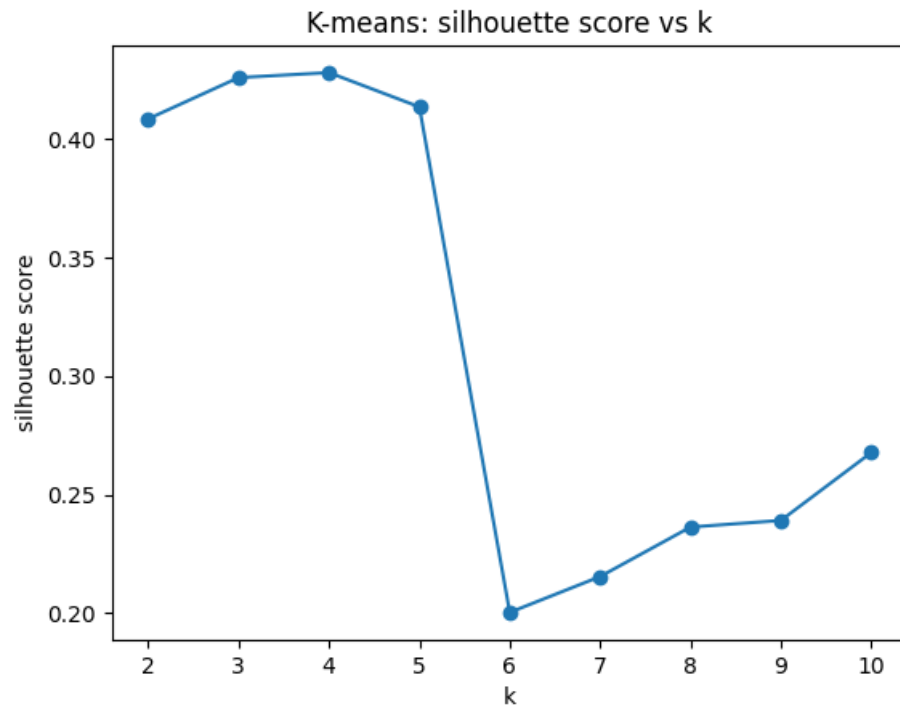


Figure 8. K-means silhouette scores across candidate values of k . The best result was obtained at $k = 4$.

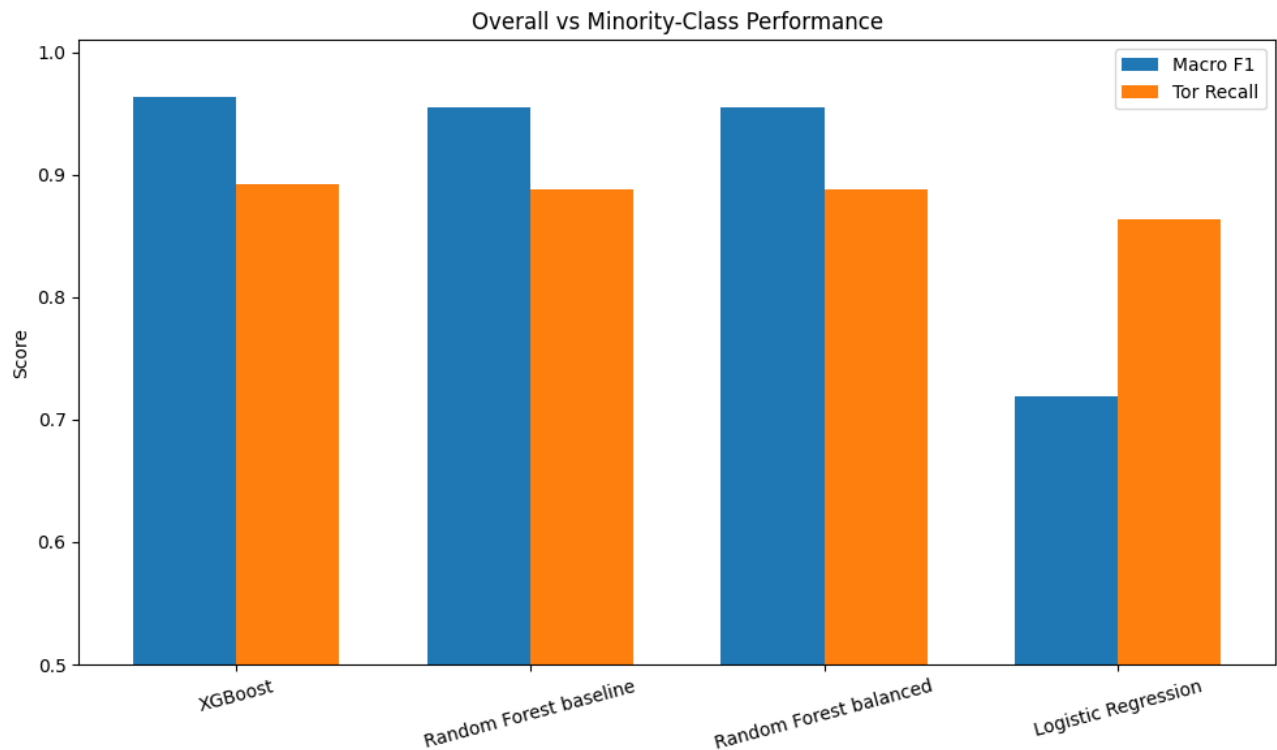


Figure 9. Overall versus minority-class performance. The gap between macro F1 and Tor recall shows why overall performance alone was not enough in this project.

5. Discussion

The results show that tree-based models were the best fit for this dataset. Both Random Forest and XGBoost clearly outperformed Logistic Regression, which suggests that the traffic classes were not separated in a simple linear way. The PCA plot supports this by showing structure in the data, but not a clean straight boundary.

Among all supervised models, XGBoost performed best. It achieved the strongest overall metrics and the strongest Tor performance. This suggests that boosted trees were better able to capture the statistical patterns in the flow features than the other models.

The baseline Random Forest was still important because it performed strongly and was easier to interpret through feature importance. For that reason, it worked well both as a baseline and as an explanatory model.

The class-weighted Random Forest was added specifically to test the Tor imbalance issue. However, it did not improve Tor precision, recall, or F1 in a meaningful way. That result still mattered because it showed that we tested the imbalance issue directly instead of only assuming that weighting would help.

The Label.1 task showed that the dataset also contains useful information for application-level classification. However, that task was clearly harder than the main Label task. This makes sense because categories such as Chat, Email, and VOIP can have more similar flow behavior than the broad traffic classes.

The clustering results added another perspective. K-means showed that the data had internal structure, while DBSCAN-related analysis suggested that Tor behaved more like an outlier than the other traffic types. This helped explain why Tor remained a special case throughout the project.

The robustness checks were also important. Strong validation scores were useful, but stricter tests showed that performance could drop when the split became harder. This suggests that some learned patterns may depend partly on source-specific behavior rather than only on the traffic label itself.

6. Limitations and Future Work

The first limitation of the project was class imbalance. Tor was the smallest class in the dataset, which made it harder to evaluate and model than the larger classes. Even though the stronger models still performed well on Tor, the smaller class size means its results should be interpreted more carefully.

The second limitation was dataset dependence. All results were based on dataset, so strong performance here does not automatically guarantee the same performance in every real-world setting.

Another limitation is that the project used flow-based features only. This was realistic for encrypted traffic analysis, but it also meant the models were limited to statistical traffic behavior. Some additional information that might help in other settings was not included.

For future work, it would be useful to test other imbalance-handling methods such as oversampling or synthetic sample generation, evaluate the models on additional traffic datasets, and study the harder Label.1 classes in more detail. We would also like to test different train-test sample like 70/30, 60/40 and so on.

7. Conclusion

This project studied whether darknet-related traffic could be classified using network flow features. The results showed that it could. Flow-based statistics were strong enough to separate the main traffic classes with very high accuracy.

Among the supervised models, XGBoost performed best, while Random Forest also gave very strong results. Logistic Regression was much weaker, which suggested that the problem was not mainly linear. The class-weighted Random Forest did not improve Tor performance, showing that simple class weighting alone was not enough to change the minority-class results.

The project also showed that Tor was the most challenging class because it had the fewest samples. Even so, the stronger models still learned Tor reasonably well. The Label.1 task showed that flow features were also useful for application-level classification, although that task was harder.

Overall, the project showed that darknet-related traffic is not invisible in flow data. It leaves statistical patterns that machine learning models can learn effectively, but those results still need to be interpreted with attention to class imbalance and generalization.

Group Contribution

Jaspinder worked on data preparation and organization, including cleaning the dataset and preparing it for modeling. He also implemented the Balanced Random Forest and XGBoost models and contributed to writing the final report.

Nomaan worked on label correction, exploratory data analysis, and understanding the class distributions in the dataset. He also helped interpret results and connect the visualizations to the report findings. He also helped compare model performance and interpret the main classification results.

Syed worked on the Random Forest baseline and PCA analysis.

References

LLM Used – ChatGPT 5.4

- [1] Pekar, A., Piny, R., & Hynek, K. (2026). Tutorial on flow-based network traffic classification using machine learning. arXiv.
- [2] Rust-Nguyen, N., Shields, K., & Choo, K.-K. R. (2022). Darknet traffic classification and adversarial attacks. arXiv.
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- [4] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the Second International Conference on Knowledge Discovery and Data Mining.